

一. 问题分析

1. 分析并确定要处理的对象是什么

本程序处理的对象是 in.txt 文件中的英文文本，程序需要从中提取出每一个英文单词进行统计。程序会忽略文本中的非字母字符（如标点符号、数字等），将其视为单词分隔符。提取单词时会统一大小写。经过这种预处理，程序获取到纯净的英文单词序列，作为后续词频统计算法的处理对象。

2. 分析并确定要实现的功能是什么

本程序的主要功能是对 in.txt 的文本进行词频统计

读取输入文本后，统计其中每个不同英文单词出现的次数。利用 Trie 树数据结构逐个单词地插入并计数，最终在 Trie 树中保存所有出现单词及其累计出现次数。

3. 分析并确定处理后的结果如何显示

按单词出现次数由高至低输出前 100 个（不足 100 个按实际个数输出），每行输出一个单词及其出现次数，单词和其出现次数间由一个空格分隔。

4. 用题目中样例，详细给出样例求解过程

以文本中这句话为例，介绍如何使用字典树进行词频统计：

Do more than others expect you to do and do it pains;

- 插入“do”：读取第一个单词"do"，Trie 树此时为空。从根结点出发，处理字符 d：根结点下无分支 d，新建结点 d。处理字符 o：在 d 结点下无分支 o，新建结点 o。在结点 o 处将 count 由 0 增至 1，表示“do”出现 1 次。
- 插入“more”：读取第二个单词"more"：根结点下新建 m 结点；在 m 下依次新建 o、r、e 结点；在 e 处将 count 设为 1，表示“more”出现 1 次。
- 插入“than”：读取第三个单词"than"：根结点下新建 t 结点；在 t 下依次新建 h、a、n 结点；在 n 处将 count 设为 1，表示“than”出现 1 次。
- 插入“others”：读取第四个单词"others"：根结点下已有 t 分支，但分支 o（在 t 下）还未创建。在 t→o 下依次新建 t、h、e、r、s 结点（形成前缀“others”）；在最后的 s 结点处将 count 设为 1，表示“others”出现 1 次。
- 插入“expect”：读取第五个单词"expect"：根结点下无 e 分支，创建 e 结点；在 e 下依次新建 x、p、e、c、t 结点；在末尾 t 结点处将 count 设为 1，表示“expect”出现 1 次。
- 插入“you”：读取第六个单词"you"：根结点下无 y 分支，创建 y 结点；在 y 下依次新建 o、u 结点；在 u 处将 count 设为 1，表示“you”出现 1 次。
- 插入“to”：读取第七个单词"to"：根结点下已有 t 分支，沿 t 继续；在 t 下无 o 子结点（此前只在 t 下创建了“than”与“others”的分支），创建 o 结点；在新建的 o 结点处将 count 设为 1，表示“to”出现 1 次。
再次插入“do”：读取第八个单词"do"：根→d→o 路径已存在；在该路径的末尾 o 结点将 count 由 1 增至 2，表示“do”第二次出现。
- 插入“and”：读取第九个单词"and"：根结点下无 a 分支，创建 a 结点；在 a 下依次新建 n、d 结点；在末尾 d 处将 count 设为 1，表示“and”出现 1 次。再次插入“do”：读取第十个单词"do"：重复第八步逻辑，将根→d→o 末尾的 count 由 2 增至 3，表示“do”第三次出现。
- 插入“it”：读取第十一个单词"it"：根结点下已有 i 分支吗？无，创建 i；

在 i 下新建 t；在 t 处将 count 设为 1，表示 “it” 出现 1 次。

- 插入 “pains”：读取第十二单词 “pains”：根下无 p，创建 p；在 p 下依次新建 a、i、n、s；最终 s 结点 count=1，表示 “pains” 出现 1 次。

词频统计结果如下

```
do 3
and 1
expect 1
it 1
more 1
others 1
pains 1
than 1
to 1
you 1
```

二. 数据结构与算法设计

1. 抽象数据类型

采用 Trie 树（字典树）作为核心数据结构来存储单词及其频率。将 Trie 树封装为一个抽象数据类型

ADT 名称：Trietree

数据对象

trie[n][26]：一个 int 二维数组用于存储从结点 p 出发、对应字母 ('a'+i) 的子结点编号；0 表示无该分支。

cont[n]：一个 int 一维数组用于记录以结点 n 作为单词结尾时该单词的出现次数；初始为 0。

id 已分配的最大结点编号，根结点固定为编号 0，每创建一个新结点则 ++id。

基本操作：

(1) 插入单词 void insert(const string &s)

1. currentNode=0（根结点初始化）

2. 对 s 中每个字符 c，计算索引 x=c-'a'

3. 若 trie[currentNode][x]==0，则 trie[currentNode][x]=++id 新建结点

4. currentNode=trie[currentNode][x]

5. 循环结束后 cont[currentNode]++，词频加 1

(2) 查询频次：int findcnt(const string &s)

1. currentNode=0

2. 逐字符计算 x=c-'a'，若 trie[currentNode][x]==0 则返回 0

3. 否则 currentNode=trie[currentNode][x]

4. 完成后返回 cont[currentNode]

(3) 提取所有词：

void extractWords(vector<pair<string,int>>& words, int node, string currentWord)

1. 若 cont[node]>0，将 (currentWord,cont[node]) 加入 words

2. 遍历 i=0...25，对每个 trie[node][i]>0 递归调用自身，前缀追加 char('a'+i)

ADT 代码如下

```
class Trietree{
public:
    int trie[n][26]; // 用二维数组存储字典树
    int cont[n]={0}; // 用数组记录每个单词的出现次数
    int id=0; // 当前结点
    // 插入单词
    void insert(const string &s) {
        int currentNode=0;
        for (int i=0;i<s.size();i++) {
            int x=s[i]-'a'; // 将字符转换为小写字母的索引
            if (trie[currentNode][x]==0)
                trie[currentNode][x]=++id; // 如果当前结点没有对应的
字母，创建新结点
            currentNode=trie[currentNode][x]; // 更新当前结点
        }
        cont[currentNode]++; // 单词插入完成，计数加 1
    }
    // 查找单词的出现次数
    int findcnt(const string &s) {
        int currentNode=0;
        for (int i=0;i<s.size();i++) {
            int x = s[i] - 'a';
            if (trie[currentNode][x] == 0)
                return 0; // 如果没有找到该单词，返回 0
            currentNode=trie[currentNode][x];
        }
        return cont[currentNode];
    }
    // 从字典树中提取单词及其频率
    void extractWords(vector<pair<string,int>>& words,int node,string
currentWord) {
        if (cont[node]>0) {
            words.push_back({currentWord,cont[node]});
        }
        for (int i=0;i<26;i++) {
            if (trie[node][i]>0) {
                extractWords(words,trie[node][i],currentWord + char(i
+ 'a'));
            }
        }
    }
};
```

2. 物理数据对象设计:

利用**静态二维数组与计数数组**的组合，通过索引来模拟结点与子结点的关系，实现了 Trie 存储

● **二维数组** `trie[n][26]`:

`trie[u][i]` 存储结点 `u` 沿字母 `'a'+i` 分支跳转到的下一级结点编号。编号 0 是根结点；如果 `trie[u][i]==0`，表示该分支不存在。

```
if (trie[currentNode][x] == 0)
```

```
    trie[currentNode][x] = ++id;
```

```
currentNode = trie[currentNode][x];
```

插入时，`++id` 分配一个新的结点索引，并写入二维数组；查找时直接读取该数组完成下跳。

● **一维数组** `cont[n]`

含义：`cont[u]` 记录编号 `u` 的结点作为单词结尾时的出现次数。只有当某条插入路径完整到达该结点时，才会执行 `cont[currentNode]++`；

在 `insert()` 的末尾累加，`findcnt()` 则直接返回 `cont[currentNode]`。

结合这两者，实现程序插入过程和查询过程的操作。

3. 算法思想的设计

程序采用 Trie 树插入与遍历的算法思想来完成单词词频统计。

核心思想是：

逐个读取单词并插入 Trie 树，在插入过程中完成计数累加；由于 Trie 树按字母前缀组织数据，通过对 Trie 的遍历可以以字典序获得所有单词及频率并输出统计结果。

算法包含两个阶段：

构建阶段：程序从输入文本读取单词，对每个单词执行插入操作。在插入单词的过程中，沿着 Trie 树逐字符走下去：如果当前字符的子结点不存在，则创建一个新结点。移动到该子结点，处理下一个字符。重复直到单词的最后一个字符。将最后一个字符对应结点的 `count` 加 1，表示该完整单词出现了一次。通过这种方式，所有单词依次插入后，Trie 树中的每个完整单词结尾结点都累积了该单词的出现次数

输出阶段：构建完成的 Trie 树可用于遍历输出所有词频，通过对 Trie 树进行深度优先遍历（DFS），可以恢复出所有插入的单词并访问它们的计数。遍历从根结点开始，递归地沿每条存在的子结点路径向下，当抵达某个结点 `count > 0` 时，即找到一个完整单词，在输出列表中加入该单词及频率。继续遍历直到所有结点都被访问。输出排序算法如下：首先比较出现次数，若出现次数相同，按字典序排序。

4. 关键步骤的算法步骤

4.1 初始化 Trie 树：创建一个 Trie 树的根结点，用于存储所有单词的共同起点。初始时根结点的所有子指针为空，计数为 0。

4.2 读取：逐字符读取输入文本，将文本划分为单词。可采用维护一个临时字符串 `word` 的方法，遍历输入：

如果读入的字符是英文字母（`isalpha(ch)` 为真），则将其转为小写后附加到临时字符串 `word` 末尾，继续读取下一个字符。

如果读入非字母字符（如空格、标点或换行），则视为单词边界。此时若临时字符串 `word` 非空，说明一个完整单词已读取完毕，调用插入操作将该单词加入 Trie 树，然后清空 `word` 准备读取下一个单词。

重复上述过程直到输入结束。读完后如果 word 仍非空（文本可能不以非字母结尾），也需将其插入 Trie 树。

4.3 插入

4.3.1. 首先将 currentNode 设为 0，即指向 Trie 树的根结点。

4.3.2. 逐字符遍历输入字符串 s：

对当前字符 c 计算索引 $x=c-'a'$ ，将字母映射为 0 到 25 之间的整数。

检查 $\text{trie}[\text{currentNode}][x]$ ，如果其值为 0，说明当前结点没有对应的子结点，需要新建：

a. 将 id 自增 1 得到新结点编号。

b. 将 $\text{trie}[\text{currentNode}][x]$ 赋值为此新编号，在数组中建立分支。

4.3.3 将 currentNode 更新为 $\text{trie}[\text{currentNode}][x]$ ，沿刚才创建或已存在的分支向下移动。

4.3.4. 遍历结束后，currentNode 指向单词最后一个字符所在的结点，执行 $\text{cont}[\text{currentNode}]++$ ，将该结点的计数加 1，完成一次词频累加。

4.4.

遍历 Trie 输出结果：所有单词插入完成后，调用遍历函数对 Trie 树进行深度优先遍历。调用提取字典树中的所有单词及其频率函数与排序函数，输出前 100 个单词及其出现次数

三．算法性能分析

1. 时间复杂度 $O(N)$

● 文件读取

按字符逐个读取，总共执行 N 次 $\text{file.get}(c)$ ，每次判定 $\text{isLetter}(c)$ 都是 $O(1)$ 操作。

对于长度为 1 的单词，插入过程遍历该单词的每个字符，执行一次数组索引和可能的一次结点分配，时间为 $O(1)$

文本中共插入 M 个单词，总时间复杂度为 $O(N)$

● 从 Trie 中提取所有单词及其频率

深度优先遍历整个 Trie，访问每个已分配结点一次。结点总数不超过插入时创建的结点数，而结点数 $\leq$ 总字符数 N

在每个结点上做常数操作（检查 $\text{cont}[\text{node}]$ 、循环 26 次判断子结点）。时间： $O(\text{结点数} \times 26) = O(N)$

● 排序与输出

wordCount 的大小即不同单词数 U

排序复杂度为 $O(N \log N)$ 。

此后只输出前 100 条记录，时间 $O(100)$

综上所述，若最坏情况下 $U \approx N$ （几乎每个单词都不同），则上界为 $O(N \log N)$ ；在典型英文文本中 $U \ll N$ ，程序近似线性 $O(N)$ 。

故时间复杂度为 $O(N)$

2. 空间复杂度 $O(N)$

程序主要内存开销来源于：

静态数组

$\text{trie}[n][26]$ ： $26 \times n$ 个整型槽位 $\rightarrow O(n)$

$\text{cont}[n]$ ： n 个整型 $\rightarrow O(n)$

`vector<pair<string,int>>` : 存放 U 个不重复单词及其字符串内容
总字符数 $S \leq N \rightarrow O(U+S) = O(N)$

递归栈深度

最多按最长单词长度 L 递归 $\rightarrow O(L)$ ($L \ll N$)

综上总空间复杂度为

$$O(n) + O(N) + O(L) \approx O(N)$$